AURA BEAT: SOFTWARE SPECIFICATION & SYSTEM ANALYSIS REPORT

Complete Technical Project Dossier

Subject: Formal Technical Onboarding Report for Full-Stack Capstone Developers
Scope: Complete Source Code Audit & Engineering Specifications

1. Project Overview & Scope Specifications

Project Title: AuraBeat: Emotion-Aware Music Generator & Voice Assistant.
Project Objective: Implement an interactive full-stack app which uses client-side computer vision models, browser speech synthesizers, and backend sentiment extraction to adapt music playing lists to users' emotional states.
Problem Statement: Traditional music providers require constant user screen interaction and manual search inputs. This isolates playback loops from the user's emotional and physical environment.
Existing System Details: Streaming platforms (like Spotify or YouTube Music) supply prepackaged playlist buckets, which remain static until the user manually switches albums.
Limitations of Existing System: High screen-time demands, lack of accessibility for physically restricted users, zero feedback on biometric visual emotions, and CORS blocking risks for developers integrating external audio source APIs.
Proposed System: AuraBeat combines real-time expression detection (face-api.js), NLP voice controllers (Gemini API), and an HTTP audio reverse-proxy (Spring Boot) backplane to automate hands-free music matches without origin CORS errors.
Benefits of the Proposed System: Zero screen interaction via STT/TTS loops; local, secure user credential validation; off-loading AI prompt expenses using multi-layered keyword fallback parsers; and visual metric displays.
Target Users: General desktop music listeners, visually or physically restricted users, and developers exploring client-side neural metrics integrations.
Project Scope: Integrates a Java 21 Spring Boot core, a Vite + React 19 Frontend single page application (SPA), and a Python Flask sentiment Classifier.
Project Goals: Real-time facial scan response under 1.5s, 100% CORS-free playback streaming, session-level memory retention via relational DB storage, and robust error recovery.
Real-world Applications: Hands-free music playback in cars, accessibility players for visually restricted users, and mood-adaptive tracks in wellness suites.
Future Scope: Miginating face model processing offline to Edge devices, integrating heart rate metrics via smartwatch APIs, and distributed caching in Redis.

2. Detailed Project Workflows

A. User Flow: The user opens the React client $\rightarrow$ authenticates via Login $\rightarrow$ hits Dashboard $\rightarrow$ either verbalizes searches, types message in chat, or scans face expression $\rightarrow$ system announces actions $\rightarrow$ matching music plays.
B. Request Flow: Frontend Axios Client resolves payload forms $\rightarrow$ sets authorization header with bearer tokens $\rightarrow$ hits Backend Spring Security filters $\rightarrow$ resolves endpoints controller handlers.
C. Backend Flow: Controller maps REST requests to target service (e.g. ChatService) $\rightarrow$ retrieves logs $\rightarrow$ evaluates token parameters $\rightarrow$ calls Gemini API for semantic classifications $\rightarrow$ resolves matched songs lists $\rightarrow$ returns unified JSON arrays.
D. Database Flow: Calls UserRepository to retrieve logged user mappings $\rightarrow$ updates EmotionHistory registries $\rightarrow$ fetches matches via `SongRepository.findByEmotion()` using Spring Data hibernate bindings.
E. Python / AI Flow: Flask app receives chat strings at `/api/analyze` $\rightarrow$ loads a DistilRoBERTa model pipeline $\rightarrow$ outputs emotion lists (Neutral/Joy/Anger/etc.) $\rightarrow$ falls back to word match scripts if host GPU/RAM hits limits.
F. Response Flow: AI engine formats instructions $\rightarrow$ client context speaks text outputs using browser SpeechSynthesis $\rightarrow$ updates music play queues $\rightarrow$ changes tracks.

3. System Architecture Specification

AuraBeat implements a **decoupled, distributed layered architecture** with a central MVC backend and strategy-driven music source provider interfaces.

[Client Web Browser] React 19 SPA (Main Viewport, Web Audio Player, Webcam Frame Scanner, Speech Rec Hooks)

| Axios HTTP JSON Requests (JWT Bearer Token Bearer Authorization Header)

[Backend Spring Boot Framework] REST Controller Layer (AuthController, ChatController, SongController, AudioStreamController)

| Business & AI Integration Service Layer

+--> **GeminiService** (HTTPS POST Request JSON -> Google Gemini 2.5 Flash API endpoint)
+--> **CatalogService** (aggregates providers: LocalProvider, SaavnProvider, AudiusProvider)
+--> **EmotionService** (tracks and parses user sentiments; contains Local Keyword Fallback Engine)
+--> **AudioStreamController** (Reverse Proxy pipelines raw audio bytes from Saavn CDN to bypass CORS)

| Hibernate ORM JPA Interfaces

[Database Storage] MySQL Server (Identified schema instance: *music_memory* containing 7 relational tables)

| Asynchronous REST Fallback calls (Port 5000)

[Python AI Sentiment Engine] Flask Microservice (Runs local emotion classification using DistilRoBERTa Transformers pipeline)

Responsibilities of each layer:

- **Client SPA**: Manages UI, captures mic and webcam input, runs client-side ML models, and resolves proxy streams.
- **API Controllers**: Validates input, checks JWT headers, and handles HTTP response types.
- **Service Layer**: Contains the core business logic, API wrappers, caches, and fallbacks.
- **Repository/JPA**: Standardizes Hibernate database interactions without manual SQL creation.
- **Python Microservice**: Evaluates expressions locally if Google Gemini API hits rate limits or is offline.

4. Folder Structure & Subdirectory Mappings

The project is divided into four main folders. Below is their purpose and interactions:

Directory Path	Key Files Inside	Purpose and Responsibilities
frontend/src/context	MusicContext.jsx, AIContext.jsx	Manages global application state, web audio references, and AI action command mappings.
frontend/src/hooks	useSpeechRecognition.js	Microphone speech recognition interface. Solves Chrome's speech recognition truncation bug.
frontend/src/pages	Dashboard.jsx, Register.jsx, Login.jsx, Playlist.jsx	Handles page styling, webcam expression capture, custom playlists creation, and user dashboard metrics.
backend/app/src/main/.../config	SecurityConfig.java, JwtAuthenticationFilter.java, JwtTokenProvider.java	Defines CORS credentials, intercepts Bearer tokens, and sets security configuration.
backend/app/src/main/.../controller	AuthController.java, SongController.java, ChatController.java, AudioStreamController.java	REST controllers. Handle client requests, JSON operations, and audio CORS bypass streaming.
backend/app/src/main/.../service	GeminiService.java, CatalogService.java, EmotionService.java, UserService.java	Handles the core business logic, third-party API integration, keyword fallbacks, and user authentication.
ai-service	app.py, requirements.txt	Local Python sentiment microservice. Classifies text locally using transformers.
database	schema.sql, data.sql	MySQL database initial configuration scripts.

5. Frontend Technical Audit (React/Vite)

AuraBeat's frontend is a single-page application built on **React 19, Vite 8, and Tailwind CSS v4**. It is fully responsive, and optimizes audio playback using a programmatic context wrapper.

Feature Implemented	Key Files Involved	Technical Working & Integration Flow
Audio Playback & Queue Manager	MusicContext.jsx	Programmatically controls a DOM-level Audio instance. Implements shuffle and repeat loops, and dynamically routes CDN URLs through a local backend proxy to bypass CORS restrictions.
Real-time Face expression scanner	Dashboard.jsx	Loads face-api.js dynamically in the browser. Uses TinyFaceDetector to classify camera frames. Employs a 7-reading voting method to filter out classification noise.
VibeBot AI Voice Assistant	AIContext.jsx, useSpeechRecognition.js	Transcribes audio via useSpeechRecognition. Evaluates response strings, and uses browser speechSynthesis to read responses aloud to the user.
Custom Playlist Mixer	Playlist.jsx	Enables custom playlist creation and tags songs with specific emotions. Users can download their playlist configurations as JSON documents.
Stateless API Client	services/api.js	Axios instance configured with interceptors. Automatically injects authorization headers, and redirects users to /login if a response returns a 401/403 status.

6. Backend Technical Audit (Spring Boot)

The Java backend runs on **Java 21 and Spring Boot 4**. It is modular and handles data retrieval, security filters, API resolution, and third-party wrappers.

Component	Implements	Responsibility & Work Flow
Security Config	SecurityConfig.java, JwtAuthenticationFilter, JwtTokenProvider	Disables CSRF, checks for JWT bearer tokens in request headers, validates signatures using HMAC-SHA keys, and populates SecurityContextHolder.

Audio Controller	AudioStreamController.java	A reverse proxy for audio streaming. Resolves external audio URLs, fetches the byte stream, and flushes output response buffers with CORS-permitted headers.
Music Catalog Service	CatalogService.java, SaavnProvider.java	Combines search results from the local database and third-party APIs. Caches search queries for 5 minutes and streaming URLs for 1 hour to prevent redundant API queries.
AI Processing Service	ChatService.java, EmotionService.java, GeminiService.java	Queries Gemini 2.5 Flash for emotional classification and action parsing. Falls back to a local keyword-matching engine if the Gemini API is offline.

7. Database Schema & Relational Specifications

AuraBeat utilizes **MySQL** as its database engine. Below is the relational mapping of the `music_memory` schema:

```
[users] (id PK | name | email UNIQUE | password | created_at)
        | 1          | 1              | 1
        | N          | N              | N (Cascade on delete)
  [playlists]    [favorites]    [emotion_history] / [chat_history]
        | 1          | N                    (user_id FK references users.id)
        | N (playlist_songs Junction table: playlid_id FK, song_id FK)
        | 1
[songs] (id PK | title | artist | emotion INDEX | genre | song_url | thumbnail | language)
```

Important Relational Mappings:
• **playlist_songs**: A junction table that maps composite key `(playlist_id, song_id)` to enable a Many-to-Many relationship between playlists and songs.
• **favorites**: Enforces a multi-column unique constraint `(user_id, song_id)` to prevent users from adding duplicate likes to the same song.
• **On Delete Cascade**: Restrict constraints are configured as Cascade. If a user deletes their profile, their playlists, favorite mappings, and emotion logs are automatically deleted.

8. Python & Artificial Intelligence Report

AuraBeat employs **Google Gemini 2.5 Flash** for sentiment analysis and chatbot actions, with a local Python microservice as a fallback.

A. Google Gemini Service:
Requests are formatted as system instructions and sent to Gemini: *Classify user intent and return structured commands only: [ACTION:navigate] [PARAM:route], [ACTION:play_song] [PARAM:emotion query].* Java's HttpClient sends a POST request with the user's message and history log, and the response is parsed using Jackson ObjectMapper nodes.

B. Python Flask Service (ai-service/app.py):
Exposes a local Flask API on port 5000. It uses Hugging Face's pipeline model (`j-hartmann/emotion-english-distilroberta-base`) to classify sentiment into 7 emotions (Joy, Sadness, Anger, Fear, Surprise, Disgust, Neutral). To prevent application failure in memory-restricted environments, it contains a local keyword regex matcher fallback to guess the sentiment from key terms (e.g. relax, sad, angry).

C. Browser voice controls:
Client speech transcriptions are captured using `webkitSpeechRecognition`. Short, silent pauses cause standard speech APIs to terminate. AuraBeat resolves this by disabling continuous speech recognition and restarting automatically after each transcription using a component reference check (`shouldRestartRef.current`).

9. Dependency Audit & Possible Alternatives

Dependency	Environment	Project Utilizations	Possible Alternatives
framer-motion	frontend	Smooth dashboard transitions & modal fade-outs.	React spring / CSS keyframes
axios	frontend	API HTTP client. Automatically injects authorization headers.	Native Fetch API
JJwt (jjwt-api)	backend	Generates and validates JWT tokens.	Auth0 Java-JWT
Spring Security	backend	Secures REST controllers using statless filters.	Apache Shiro / Custom filters
transformers	python	Loads DistilRoBERTa model pipeline.	NLTK / TextBlob Sentiment

10. API Documentation (AuraBeat REST endpoints)

API Endpoint Route	Method	Controller Layer	Authentication?	JSON Request/Response Payload Types
/api/register	POST	AuthController	No	Req: {name, email, password} Res: jwtToken, user details
/api/login	POST	AuthController	No	Req: {email, password} Res: jwtToken, user details
/api/chat	POST	ChatController	Yes (Bearer)	Req: {message} Res: botResponse, detectedEmotion, recommendedSongs
/api/emotion	POST	EmotionController	Yes (Bearer)	Req: {text} Res: emotion (e.g. Happy, Sad)
/api/songs	GET	SongController	Yes (Bearer)	Params: emotion, genre, search, language Res: Array of song objects
/api/audio/stream	GET	AudioStreamController	No	Params: url (JioSaavn CDN path) Res: Raw audio bytes pipeline output

11. Security Implementation Report

AuraBeat implements a **stateless security design** using Spring Security. Below is the details of this workflow:
• **Authentication Filter**: The `JwtAuthenticationFilter` intercepts requests. It parses the authorization header, verifies token signatures, and populates Spring's context details.
• **Password Security**: Passwords are encrypted before database commit using `BCryptPasswordEncoder` (configured with a work factor of 10). matches are validated via: `encoder.matches(raw, hashed)`.
• **CORS (Cross-Origin Resource Sharing)**: Backend controllers allow browser resource requests via `@CrossOrigin(origins = "*")` annotations. To avoid CORS restrictions when streaming, audio files are proxied through local backend ports.

12. Deployed Algorithms & Logic

A. Webcam Recognition Smoothing Algorithm:
Raw webcam detections can struggle with expression flickering. AuraBeat resolves this using a **queue smoothing filter**:
1. Real-time predictions are captured from the webcam via face-api.js every 1.2 seconds.
2. Predictions with a confidence score under 0.40 (or 0.60 for Neutral) are discarded.
3. Decisive predictions are added to a queue containing the last 7 readings.
4. A majority vote is calculated over the queue once it contains at least 3 entries, determining the user's emotion.
5. The system commits the detected emotion only if it differs from the previously logged emotion, reducing duplicate API calls.

B. Catalog Caching & Deduplication Algorithm (CatalogService):
Aggregates search results dynamically from the local database, JioSaavn Dev API, and Audius APIs. To prevent rate limits and improve response times, it caches search queries for 5 minutes and streaming URLs for 1 hour. It deduplicates tracks in $O(N)$ using lowercase Title + Artist comparisons. Local database results are prioritized and displayed first.

13. Software Design Patterns in Practice

• **Proxy Pattern**: Implemented in `AudioStreamController.java` to act as an HTTP reverse proxy. It fetches external audio bytes from CDNs and streams them to bypass CORS restrictions.
• **Strategy Pattern**: Reflected in the `MusicProvider` interface. Switchable implementations (`SaavnProvider`, `AudiusProvider`, `LocalProvider`) execute dynamically under `CatalogService` based on available APIs.
• **Singleton Pattern**: Implemented by Spring Boot's Enclosing IOC Container, which handles service initializations (e.g. `GeminiService`).
• **Observer Pattern**: Implemented in `MusicContext` to monitor HTML5 audio player properties and push updates to the UI.

14. System Configuration Reports

• **application.properties**: Configures the MySQL connection string (`spring.datasource.url=jdbc:mysql://localhost:3306/music_memory`), metadata properties (`jpa.hibernate.ddl-auto=update`), JWT secret keys, and `gemini.api.key`.
• **vite.config.js**: Handles compilation and React 19 plugins.
• **package.json**: Manages dependencies (e.g. Framer Motion, Axios, Tailwind CSS v4, Lucide Icons).
• **pom.xml**: Manages Maven dependencies (e.g. Spring Security, Spring Boot Web, Spring Data JPA, Hibernate, JJWT).
• **requirements.txt**: Configures python dependencies (Flask, Flask-CORS, Transformers, Torch, Sentencepiece, Werkzeug).

15. Component Execution Flow

1. Application Startup: Spring Boot initialises, reads configurations, validates the database connection, and database seed scripts map initial entries. The Vite React client starts up, checks for cached user credentials in local storage, and redirects the user to the Login page.
2. User Login: User submits their email and password $\rightarrow$ the login controller validates credentials via BCrypt $\rightarrow$ a JWT is generated and returned to the client $\rightarrow$ the client saves the token in local storage and redirects the user to the Dashboard.
3. Speech Request Processing: User speaks a query $\rightarrow$ client transcribes it using `useSpeechRecognition` $\rightarrow$ Axios sends a POST request with the transcription to `/api/chat` $\rightarrow$ the backend parses the query's emotion and intent using the Gemini API $\rightarrow$ matching songs are queried and returned to the client $\rightarrow$ the browser speaks the response aloud and plays the matching tracks.

16. System Error Handling & Resilience

• **Frontend:** Axios response interceptors catch 401/403 errors, clear local storage, and redirect users to log in if their token has expired. The Dashboard includes a retry banner if catalog API requests fail.
• **Backend REST Exception Handlers:** Controllers capture Exceptions and return appropriate HTTP status codes (e.g. 400 Bad Request, 502 Bad Gateway).
• **AI Fallbacks:** If the Gemini API is offline or returns an error, `EmotionService` and `ChatService` fall back to local keyword pattern matching to parse queries.

17. System Performance Optimization Features

• **Caching:** `CatalogService` caches search queries for 5 minutes and streaming URLs for 1 hour using Java `ConcurrentHashMap` to reduce external API queries.
• **Client-side computations:** Facial recognition is processed locally in the browser using the user's GPU/CPU. Webcam snapshots do not need to be uploaded to the server.
• **Consolidated Aggregation:** Caching search queries and deduplicating tracks locally results in catalog search response times under 400ms.

18. Project Uniqueness Summary Table

Feature Component	Technical Uniqueness	Value Added
Auditory UI Loop	Combination of Text-to-Speech synthesis and Speech-to-Text command tracking.	Improves accessibility for users with limited mobility or vision impairments.
Adaptive Playback	Aggregates search results dynamically from local databases and streaming APIs.	Increases the size of the music catalog without expanding database storage.

19. Technical Challenges & Solutions Implemented

Discovered Challenge	Technical Solution Implemented	Source Files Involved	Key Benefit
CORS blocking external audio streams	Implemented a reverse proxy controller to pipe raw audio bytes locally.	AudioStreamController.java, MusicContext.jsx	Ensures CORS-free playback of external CDN audio tracks.
Chrome Speech Recognition silent truncation	Disabled continuous mode and auto-restarts recognition on end.	useSpeechRecognition.js	Resolves microphone recording limitations.
Facial scan flickering expressions	Implemented a 7-reading voting queue before committing emotions.	Dashboard.jsx	Improves emotion classification accuracy.

20. Project Evaluation: Strengths & Weaknesses

Strengths: Built-in offline fallbacks, client-side face scanning that saves network bandwidth, and a secure JWT authentication system.
Weaknesses: Caching uses local server memory instead of a distributed cache, and speech recognition relies on browser support and internet access.
Recommended Scalability Improvements: Use Redis to manage cached queries, and run a localized OpenAI Whisper compiler offline to handle speech recognition.

21. Technical Stack Inventory Details

Discovered Tech	Category	Purpose / File Implementations	Advantage
React 19 / Vite 8	Frontend Core	App.jsx, Components, Pages	Fast compilation and rendering
Spring Boot 4 / Java 21	Backend Core	pom.xml, src/main/java	Type-safe compilation
MySQL	Database	database/schema.sql	Stable relational structure
Google Gemini API	LLM	GeminiService.java	Advanced conversational reasoning
face-api.js	Computer Vision	Dashboard.jsx	Serverless computer vision

22. Final Project Executive Summary

AuraBeat demonstrates how to integrate client-side biometrics and cloud NLP to automate music selection. The project features a clean, responsive layout built on React 19 and Vite. The backend runs on Java 21 and Spring Boot, utilizing Spring Security for authentication and Hibernate JPA to manage database interactions. AuraBeat integrates Google Gemini for conversational input and emotion detection, with offline fallbacks to handle rate limits and network issues. The caching mechanism and reverse audio proxy structure successfully deliver a CORS-free music streaming experience.